Dash0

HACKATHON CHALLENGE

Breaking Observability Vendor Lock-In

Build an automated migrator that moves dashboards, queries and alerts from Datadog into Dash0, using open standards: Perses, PromQL and OpenTelemetry.

01 Dash0 Hackathon Challenge: Breaking Observability Vendor Lock-In

02 Business Impact: Reducing Vendor Lock-In and the Cost of Observability Migration

03 Dash0 Observability Dashboard and Alert Migrator

01 THE CHALLENGE

Dash0 Hackathon Challenge: Breaking Observability Vendor Lock-In

The Problem

Proprietary observability platforms create significant vendor dependency by making it difficult and expensive for organizations to migrate between providers.

Over time, companies invest considerable engineering resources building dashboards, monitoring queries, alert configurations, and operational workflows within their observability platforms. These assets become tightly coupled to proprietary data structures, query languages, and visualization formats.

Even when a competing platform offers better pricing, functionality, or flexibility, the cost and complexity of migrating existing observability configurations can become a major barrier to switching.

Organizations are often forced to spend weeks or months manually recreating dashboards, rewriting queries, rebuilding alerts, and validating that critical monitoring capabilities have not been lost.

This creates substantial engineering costs, delays platform adoption, and reinforces vendor lock-in.

Why This Is Technically Challenging

This is not a problem that can simply be solved by throwing an LLM at it.

Although LLMs can interpret query syntax and suggest alternative implementations, they cannot reliably guarantee semantic equivalence between proprietary query languages, metric definitions, alert conditions, and visualization behaviors.

A query that looks syntactically correct after translation may produce entirely different results. Similarly, an incorrectly translated alert could result in missed incidents or false positives.

Reliable migration requires understanding the underlying semantics of observability configurations, identifying equivalent functionality across platforms, handling unsupported features, and validating that migrated configurations preserve their original intent.

The Challenge

Build a solution that automates the migration of observability dashboards, queries, and alerts from proprietary platforms into Dash0.

The solution should demonstrate the ability to:

- **Ingest existing configurations:** Retrieve observability dashboards, queries, and alerts through platform APIs or exported configuration files.
- **Translate configurations:** Convert proprietary dashboard structures, queries, and alert definitions into Dash0-compatible representations, leveraging PromQL for metric queries, Prometheus Alerting Rules + extension for alert definitions, and Perses for dashboard specifications.
- **Handle incompatibilities:** Identify unsupported widgets, query functions, and platform-specific functionality, providing alternatives or clearly identifying where manual intervention is required via logs.
- **Validate and migrate:** Demonstrate that translated configurations can be imported into Dash0 while preserving their original functionality wherever possible.

Teams are free to determine their own technical approach, architecture, and tools. AI may be used where it adds meaningful value, but simply using an LLM or wrapper to rewrite configurations is not sufficient and will be disqualified.

Expected Outcome

Deliver a working proof of concept demonstrating an end-to-end observability migration from datadog to dash0.

At a minimum, teams should demonstrate the ingestion of an existing dashboard and alert, its translation into a Dash0-compatible configuration, and the resulting dashboard and alert within the dash0 platform.

Additional Vendor Support: While Datadog should be the primary focus, solutions that support migration from other observability platforms, such as Cribl, New Relic, or other vendors, are encouraged and may receive additional points. Teams should prioritize a strong Datadog migration workflow before expanding support to additional platforms.

The Bigger Picture

The goal is to reduce the technical and financial barriers associated with switching observability providers.

By making dashboards, queries, and alerts more portable, we can reduce engineering costs, accelerate customer onboarding, preserve existing monitoring knowledge, and give organizations greater freedom to choose their observability platforms without being constrained by proprietary configurations.

Observability should not become a long-term vendor commitment simply because migrating existing monitoring infrastructure is too expensive or complex.

Business Impact: Reducing Vendor Lock-In and the Cost of Observability Migration

The Business Problem

Proprietary observability tooling creates significant vendor dependency by making it difficult and expensive for organizations to transition between providers. As companies invest years building dashboards, monitoring queries, alert configurations, and operational workflows within a particular platform, these assets become tightly coupled to the vendor's proprietary data structures, query languages, and configuration formats.

Even when a competing platform offers better pricing, functionality, or flexibility, the cost and complexity of migrating existing observability configurations can become a major barrier to switching.

Why LLMs Alone Cannot Solve This Problem

This is not a problem that can simply be solved by throwing an LLM at it. Although LLMs can help interpret query syntax, explain configurations, and suggest alternative implementations, they cannot reliably guarantee semantic equivalence between proprietary query languages, metric definitions, alert conditions, and visualization behaviors.

Observability configurations often contain complex dependencies, vendor-specific functions, implicit assumptions, and platform-specific behaviors. A query that looks syntactically correct after translation may produce different results, while an incorrectly translated alert could result in missed incidents or false positives.

Reliable migration requires more than generating syntactically valid configurations. It requires deterministic parsing, explicit translation rules, schema mappings, validation, and an understanding of the capabilities and limitations of both the source and destination platforms.

The Financial and Operational Impact

The switching costs associated with proprietary observability platforms extend far beyond the price of the platform itself. Organizations must dedicate engineering resources to manually recreate dashboards, rewrite queries, rebuild alerts, validate configurations, and ensure that critical monitoring capabilities are not lost during the transition.

This diverts engineering time away from core product development, increases implementation costs, extends migration timelines, and introduces operational risks, particularly when migrating monitoring systems responsible for identifying production incidents.

Automating this process creates tangible business value by:

- **Reducing engineering costs:** Minimizing the engineering hours required to manually recreate and validate observability configurations.

- **Accelerating customer onboarding:** Shortening the time between adopting a new observability platform and achieving operational readiness.
- **Reducing migration risk:** Preserving existing monitoring capabilities and identifying unsupported functionality before it creates operational blind spots.
- **Preserving operational knowledge:** Allowing organizations to retain the dashboards, alerts, and monitoring workflows they have spent years developing.
- **Reducing vendor lock-in:** Making it easier for organizations to evaluate and transition between observability providers without being constrained by proprietary configurations.

The Broader Opportunity

Ultimately, the objective is to reduce the switching costs associated with proprietary observability platforms, give organizations greater freedom to choose their technology providers, and make migrating to open, standards-based observability platforms a significantly more accessible and economically viable decision.

By automating the migration of dashboards, queries, and alerts, we can transform what is traditionally a resource-intensive engineering project into a repeatable and scalable process.

Dash0 Observability Dashboard and Alert Migrator

Project Overview

Dash0 is proposing a hackathon project to build an automated observability dashboard and alert migrator. The goal is to simplify migration from proprietary observability platforms, such as Datadog, Cribl and New Relic, to Dash0 by eliminating the need to manually recreate dashboards, rewrite queries, and rebuild alerts.

The proposed solution is a CLI-based transpiler that uses deterministic, rule-based translations rather than LLMs to translate dashboards, queries, and alerts between observability platforms.

Proposed Functionality

1. Connect to Existing Observability Platforms

The CLI will allow users to select their source observability vendor, initially supporting platforms such as Datadog Cribl and New Relic.

Using the source vendor's official APIs and user-provided credentials, the tool will retrieve the organization's existing dashboards, widgets, queries, and alert configurations. Another option would be to allow users to specify a folder for the tool to ingest dashboards from. For example, users could export their Datadog dashboards as YAML files and point the tool to the folder containing those exports.

2. Compile and Translate Dashboards

The migrator will parse the retrieved dashboard definitions and translate them into Dash0-compatible dashboard configurations using Perses dashboard specification.

This process will include:

- Translating vendor-specific dashboard and widget structures into corresponding Perses-compatible components that are supported.
- Preserving dashboard layouts, widget configurations, titles, and descriptions wherever possible.
- Translating proprietary query syntax into PromQL or the appropriate Dash0-compatible query representation.
- Mapping proprietary metric names and attributes to corresponding OpenTelemetry semantic conventions and standard metric names wherever applicable.

The objective is to produce portable, non-proprietary dashboard definitions that can be imported into Dash0 without manually rebuilding each dashboard.

3. Compile and Translate Alerts

In addition to dashboards, the migrator will retrieve and translate existing alert configurations.

This includes converting vendor-specific alert definitions, conditions, thresholds, evaluation intervals, and supported notification settings into equivalent Dash0-compatible configurations (Prometheus Alerting rules have an extension for thresholds).

4. Handling Unsupported Widgets and Queries

Where an exact equivalent does not exist, the transpiler should flag unsupported functionality within its logs.

5. Import into Dash0

Once compilation is complete, the CLI will connect to Dash0 using its supported APIs or import mechanisms and create the translated dashboards and alerts within the destination organization.

The tool should provide clear feedback on successful imports, unsupported features, query translation errors, and configurations requiring manual intervention.

Utilize these API's:

Import Dashboard

Import Check Rule

Technical Architecture

The proposed workflow is:

1. **Source selection:** The user selects an observability vendor and supplies the necessary API credentials.
2. **Asset discovery:** The CLI retrieves dashboards and alerts through the source vendor's APIs.
3. **Parsing:** The transpiler converts vendor-specific configurations into an intermediate representation.
4. **Deterministic translation:** Rule-based transformations map dashboard structures to Perses, queries to PromQL where feasible, and proprietary metric names / attributes to OpenTelemetry metric naming and semantic conventions where applicable.
5. **Validation:** The transpiler identifies unsupported features, ambiguous mappings, and invalid or incomplete translations for each widget.
6. **Destination import:** The CLI uses Dash0's supported interfaces to create the translated dashboards and alerts.

7. **Migration report:** The CLI summarizes successfully migrated assets, warnings, and widgets that require manual intervention.

Core Design Principles

- **No LLM dependency for query translation:** Query and configuration transformations must be performed by deterministic transpiler rules, parsers, and explicit mappings.
- **API-driven migration:** Source discovery and destination import should use official platform APIs and supported interfaces.
- **Open standards:** Use Perses for dashboard representation, OpenTelemetry semantic conventions for telemetry naming, and PromQL as the target query language where applicable.
- **Extensible vendor support:** Design the architecture so additional source observability platforms can be supported through vendor-specific adapters.
- **Transparency and correctness:** Report unsupported constructs and preserve the original configuration for reference instead of silently discarding functionality.
- **Practical usability:** Make the migration process executable through a straightforward CLI workflow.

Expected Outcome

The hackathon deliverable should be a working proof of concept that can connect to at least one source observability platform, retrieve existing dashboards and alerts, compile supported configurations into Dash0-compatible formats, and import the results into Dash0.

The broader objective is to make observability platform migration significantly faster and more reliable while reducing vendor lock-in through open dashboard formats, standardized telemetry conventions, and portable query representations.

Submission Instructions

Please use the following repository as the starting point for your work:

https://github.com/vimouzin/dash0-dashboard-migrator

1. Fork the repository to your own GitHub account.
2. Clone your fork and create a dedicated feature branch for your contribution.
3. Complete all development work on that branch. Do not commit directly to the original repository's main branch.
4. Push your changes to your fork.
5. Open a Pull Request (PR) from your feature branch against the original repository's main branch.

The PR should include a clear summary of your implementation, key features, setup and usage instructions, and any known limitations. Please ensure the solution can be reviewed and tested by the Dash0 team.

Account Setup

During the hackathon you may need an account on one or both observability platforms. Both are free to try and take only a few minutes to set up. Please do this before your team starts building.

Dash0

1. Go to **accounts.dash0.com/sign-in** and click **Sign up** at the bottom of the page.
2. Sign up with your Google account, your GitHub account, or an email address. If you use email, you will receive a verification code. Enter it to finish signing up.
3. You will be asked to name your organization and choose a data storage region. **Your organization name must start with "thinc"**, followed by a name of your choice, for example **thinc_rocketteam** or **thinc_teamblue**. Pick the EU region.

Datadog

1. Go to **datadoghq.com** and click **Free Trial** (or "Get started free").
2. Before signing up, choose your site (region). Pick **EU** so you land on app.datadoghq.eu. Important: the region cannot be changed later, and you always need to log in on the same site.
3. Enter your email and use your team name as the company name. The trial runs for 14 days and needs no credit card.
4. After signing up, you can skip the stack questions. Your API key can be found under Organization Settings, API Keys.

Tips

- One account per team is enough. The person who creates it can invite teammates from the organization settings.
- If you get stuck, ask one of the mentors.

Ready Made Datadog Content for Testing

The repo **github.com/DataDog/integrations-core** contains Datadog's official assets for almost 300 integrations (Redis, Postgres, Kafka, NGINX, Kubernetes and many more). For each integration you'll find ready made dashboards, monitors (Datadog's equivalent of check rules), log pipelines and saved views as JSON files in the assets folder. You can import them into your Datadog org to test with or to use as a starting point.